Linked List, Stack and Queue — Student Reference Module

CpE 211 — Data Structures and Algorithms

This module is your reference guide for **Linked Lists**, **Stacks** and **Queues**. The first half shows how to build a chain of connected nodes and how to traverse, insert into, delete from, and search it. The second half shows how a Stack and a Queue are built on top of that same chain. Every example is written in Python and can be run exactly as shown. Keep this handy during labs.

> **How to use this module.** It works for both of us. Students follow the code and the **Line by line** lists; the 🎤 **Instructor notes** give you the questions to ask and the points to stress. Run each code block live when you can.

Session plan (90 minutes)

Time	Topic	Sections
0:00–0:05	Class & object refresher (skip if the class module is fresh)	1
0:05–0:12	What is a linked list, and the Node	2
0:12–0:40	Traversal, insertion, deletion, searching	3–7
0:40–1:00	Stack (LIFO)	8–10
1:00–1:20	Queue (FIFO)	11–13
1:20–1:30	Compare, wrap-up, questions	14–15

If you run short on time: trim Section 1, then `insert_at_position` and `delete_at_position`. Keep traversal, insert at beginning and end, delete from beginning, push/pop and enqueue/dequeue.

Table of Contents

Part 1: Linked List

1. Class & Object Refresher
2. What is a Linked List?
3. Traversal
4. Insertion
5. Deletion
6. Searching
7. Putting It All Together

Part 2: Stack and Queue

8. What is a Stack?
9. Building a Stack
10. Stack Example: Reversing a Word
11. What is a Queue?
12. Building a Queue

13. Queue Example: A Waiting Line
14. Stack vs Queue
15. Quick Reference Cheat Sheet

---

1. Class & Object Refresher

What it means: A **class** is a blueprint for creating **objects** that group related data and behavior together. Before we build a Linked List, it's worth reconnecting with this idea — because a `Node`, which we're about to define, is just another class, exactly like the ones below.

A simple class

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"{self.name}, {self.age} years old")
```

- `class Student:` — declares a new blueprint named `Student`. Nothing is created yet — this just describes what a Student *would* look like.
- `__init__` — a special method that runs automatically every time a new `Student` object is created.
- `self` — refers to **this specific object** being built or used, not any other `Student`.
- `self.name = name` — stores the value permanently on this particular object.

Creating and using an object

```
s1 = Student("Jaymark", 19)
s1.display()
```

Output:

```
Jaymark, 19 years old
```

- `Student("Jaymark", 19)` — **creates** (instantiates) an actual object from the blueprint.
- `s1.display()` — **calls a method** on that object, using dot notation.

Two objects, two separate sets of data

```
s1 = Student("Jaymark", 19)
s2 = Student("Maria", 20)

s1.display()
s2.display()
```

Output:

```
Jaymark, 19 years old
Maria, 20 years old
```

`s1` and `s2` are two **completely separate objects**, even though both were built from the exact same `Student` blueprint. Changing one has zero effect on the other — `self` is what keeps each object's data independent.

⚠ Why this matters for Linked Lists

A `Node` (coming up next) is a class too — it just happens to store a **reference to another Node** instead of a plain value like `name` or `age`. If `self` and object independence make sense here with `Student`, the `Node` class will make sense the exact same way.

Reference: Classes and Object in Programming — GeeksforGeeks

---

2. What is a Linked List?

What it means: A linked list is a chain of **nodes**, where each node holds two things: some **data**, and a **reference** (a "next pointer") to the following node in the chain. Unlike an array, the elements are not stored next to each other in memory — they're connected by these references instead.

Basic terminology

Term	Meaning
Node	A single unit in the list, holding data and a reference to the next node
Head	A reference to the **first** node in the list — this is how you find the start of the chain
Data	The actual value stored inside a node
Next	The part of a node that points to the next node — `None` if it's the last one

The Node class

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Line by line:

1. `class Node:` — declares a new blueprint called `Node`. One Node is one box in the chain.
2. `def __init__(self, data):` — the constructor. It runs automatically whenever we write `Node(...)`. `self` is the new node being built; `data` is the value we pass in.
3. `self.data = data` — saves the value inside this node.
4. `self.next = None` — a brand-new node is not connected to anything, so `next` starts as `None`. Later we replace `None` with another node.

> 🎤 **Instructor note (~2 min):** Draw one box with two compartments on the board: `[ data | next ]`. Ask: *"What is inside `next` when the node is alone?"* (Answer: `None`.) Everything else this session builds on this one box.

- `self.data` — stores the actual value.
- `self.next` — starts as `None`, meaning "not connected to anything yet."

Building a chain manually

```
head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)

temp = head
while temp is not None:
    print(temp.data, end=" ")
    temp = temp.next
```

Output:

```
10 20 30 40
```

Line by line:

1. `head = Node(10)` — creates the first node holding 10. The variable `head` now refers to it, so we can always find the start of the chain.
2. `head.next = Node(20)` — creates a second node (20) and stores it in the first node's `next`. The two are now linked.
3. `head.next.next = Node(30)` — read left to right: from `head`, go to its `next` (the 20 node), and set *that* node's `next` to a new 30 node.
4. `head.next.next.next = Node(40)` — same idea, one step further down the chain.
5. `temp = head` — `temp` is a walker that starts at the first node. We use `temp` so that `head` is never moved and the start of the list is never lost.
6. `while temp is not None:` — keep going as long as `temp` is on a real node. `None` means we walked past the last node.
7. `print(temp.data, end=" ")` — print this node's value; `end=" "` keeps everything on one line separated by spaces.
8. `temp = temp.next` — move the walker to the next node. Without this line the loop never ends.

Picture of the chain:

```
head
 |
 v
[10 | •]--->[20 | •]--->[30 | •]--->[40 | None]
```

> 🛠️ **Instructor note (~3 min):** Run this live, then ask: *"What happens if I delete the line `temp = temp.next`?"* (Infinite loop; Ctrl+C stops it.) Remind them this is the same traversal idea as the array `for` loop, except we follow `next` instead of an index.

Each `Node` is a completely separate object. `head.next = Node(20)` doesn't copy anything — it just tells the first node "the next one is this new node."

Why use a Linked List instead of an Array?

	Array	Linked List
Insert/delete at the beginning	Slow — everything shifts	Fast — just update a reference
Access by index (`arr[i]`)	Fast	Slow — must walk the chain
Memory layout	Contiguous	Scattered, connected by references

Reference: Introduction to Linked List — GeeksforGeeks

---

3. Traversal

What it means: Traversal means visiting every node in the list, one at a time, starting from the head — following each `next` reference until you reach `None`.

To keep our code organized, we'll wrap everything in a `LinkedList` class that keeps track of the `head` for us:

```
class LinkedList:
    def __init__(self):
        self.head = None

    def traverse(self):
        temp = self.head
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()
```

Line by line:

1. `class LinkedList:` — a wrapper class whose only job is to remember where the chain starts.
2. `def __init__(self):` / `self.head = None` — a new list starts empty, so `head` is `None`. "Empty list" and "head is None" mean the same thing.
3. `def traverse(self):` — a method that visits every node.
4. `temp = self.head` — the walker starts at the first node. `self.head` stays untouched.
5. `while temp is not None:` — repeat until we fall off the end of the chain.
6. `print(temp.data, end=" ")` — show the current node's value on the same line.
7. `temp = temp.next` — step forward to the next node.
8. `print()` — after the loop, print a blank line so the next output starts on a fresh line.

```
ll = LinkedList()
ll.head = Node(10)
ll.head.next = Node(20)
ll.head.next.next = Node(30)

ll.traverse()
```

Output:

```
10 20 30
```

Line by line:

1. `ll = LinkedList()` — creates an empty list (`ll.head` is `None`).
2. `ll.head = Node(10)` — the first node becomes the head.
3. `ll.head.next = Node(20)` and `ll.head.next.next = Node(30)` — attach two more nodes, exactly like the manual chain earlier.
4. `ll.traverse()` — walks from the head and prints `10 20 30`.

> 🎤 **Instructor note:** Ask: *"Why `while` and not `for`?"* (We do not know how many nodes there are. We stop when we hit `None`.)

⚠ Important: `while`, not `for`

We use a `while` loop here — not a `for` loop — because we don't know the length of the list in advance the way we do with an array (`len(arr)`). We just keep going **until `temp` becomes `None`**, which signals the end of the chain.

Reference: Traversal of Singly Linked List — GeeksforGeeks

---

4. Insertion

What it means: Insertion adds a new node into the list. As with arrays, *where* you insert changes how the operation works — but unlike arrays, nothing ever needs to "shift." We only ever update a few `next` references.

Insert at the beginning

```
def insert_at_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
```

Line by line:

1. `new_node = Node(data)` — build the new box. It is not connected to anything yet.
2. `new_node.next = self.head` — **connect first:** the new node points at the node that is currently first.
3. `self.head = new_node` — **then move the head:** the list now starts at the new node.

Trace (list is `10 → 20 → 30`, inserting 5):

```
Before:           head -> [10] -> [20] -> [30]
After line 2:     [5] -> [10] -> [20] -> [30]      (head still on 10)
After line 3:     head -> [5] -> [10] -> [20] -> [30]
```

> 🎤 **Instructor note:** Ask: *"What if lines 2 and 3 were swapped?"* If `head` moves first, `self.head` becomes the new node and `new_node.next = self.head` points the node at itself. The old chain is lost. This is the rule **connect before you disconnect.** The new node points to whatever used to be the head, and then becomes the new head itself.

Insert at the end

```
def insert_at_end(self, data):
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        return
    temp = self.head
    while temp.next is not None:
        temp = temp.next
    temp.next = new_node
```

Line by line:

1. `new_node = Node(data)` — build the new box.
2. `if self.head is None:` — special case: the list is empty.
3. `self.head = new_node` / `return` — the new node becomes the head, and we stop here.
4. `temp = self.head` — otherwise start walking from the first node.
5. `while temp.next is not None:` — keep walking *while there is a node after this one*. We stop **on** the last node, not after it.
6. `temp = temp.next` — step forward.
7. `temp.next = new_node` — the last node now points at the new node, which becomes the new last node.

Why `temp.next` and not `temp`? If we used `while temp is not None`, we would walk off the end and have no node left to attach to. Checking `temp.next` lets us stop on the last real node. We walk all the way to the last node (the one whose `next` is `None`), then attach the new node there.

Insert at a given position

```
def insert_at_position(self, data, position):
    if position == 0:
        self.insert_at_beginning(data)
        return
    new_node = Node(data)
    temp = self.head
    for _ in range(position - 1):
        temp = temp.next
    new_node.next = temp.next
    temp.next = new_node
```

Line by line:

1. `if position == 0:` — position 0 is the same as inserting at the beginning.
2. `self.insert_at_beginning(data)` / `return` — reuse the method we already wrote, then stop.
3. `new_node = Node(data)` — build the new box.
4. `temp = self.head` — start at the first node.
5. `for _ in range(position - 1):` — walk `position - 1` steps so `temp` ends on the node **just before** the target spot. The `_` means we do not need the loop counter.
6. `temp = temp.next` — one step forward each time.
7. `new_node.next = temp.next` — **connect first:** the new node points at whatever used to come after `temp`.
8. `temp.next = new_node` — **then connect the front:** `temp` now points at the new node.

Trace (list is `5 → 10 → 20 → 30`, insert 15 at position 2):

```
range(1) -> one step: temp moves from 5 to 10
new_node.next = temp.next      15 -> 20
temp.next = new_node           10 -> 15
Result: 5 -> 10 -> 15 -> 20 -> 30
```

> 🎤 **Instructor note:** This is the hardest insertion. Have students act it out: five students hold a chain of hands, a new student steps in. The newcomer takes the next person's hand **before** the previous person lets go. We walk to the node **just before** the target position, then splice the new node in between it and whatever came next.

Testing all three together

```
ll = LinkedList()
ll.insert_at_end(10)
ll.insert_at_end(20)
ll.insert_at_end(30)
print("After inserting 10, 20, 30 at end:")
ll.traverse()

ll.insert_at_beginning(5)
print("After inserting 5 at beginning:")
ll.traverse()

ll.insert_at_position(15, 2)
print("After inserting 15 at position 2:")
ll.traverse()
```

Output:

```
After inserting 10, 20, 30 at end:
10 20 30
After inserting 5 at beginning:
5 10 20 30
After inserting 15 at position 2:
5 10 15 20 30
```

Line by line:

1. The three `insert_at_end` calls build `10 → 20 → 30`.
2. `insert_at_beginning(5)` puts 5 in front: `5 10 20 30`.
3. `insert_at_position(15, 2)` puts 15 at index 2: `5 10 15 20 30`.
4. Each `ll.traverse()` prints the list so we can see the result of the step before it.

References:

- Insert at Front — GeeksforGeeks
- Insert at End — GeeksforGeeks
- Insert at a Specific Position — GeeksforGeeks

---

5. Deletion

What it means: Deletion removes a node from the list by re-routing the `next` references around it — the removed node is simply no longer connected to anything.

Delete from the beginning

```
def delete_from_beginning(self):
    if self.head is None:
        return
    self.head = self.head.next
```

Line by line:

1. `if self.head is None:` — nothing to delete if the list is empty.
2. `return` — leave the method without doing anything.
3. `self.head = self.head.next` — move `head` to the second node. The old first node is no longer reachable, so Python cleans it up.

> 🎤 **Instructor note:** One line does the whole job. Compare with `arr.pop(0)` on an array, which shifts every element. Ask: *"Which is faster for a long list, and why?"* The second node becomes the new head. The old first node still exists in memory for a moment, but nothing points to it anymore.

Delete from the end

```
def delete_from_end(self):
    if self.head is None:
        return
    if self.head.next is None:
        self.head = None
        return
    temp = self.head
    while temp.next.next is not None:
        temp = temp.next
    temp.next = None
```

Line by line:

1. `if self.head is None:` / `return` — empty list, nothing to do.
2. `if self.head.next is None:` — the list has exactly one node.
3. `self.head = None` / `return` — remove it by emptying the list.
4. `temp = self.head` — start at the first node.
5. `while temp.next.next is not None:` — keep walking until `temp` is the **second-to-last** node (the node after `temp` is the last one).
6. `temp = temp.next` — step forward.
7. `temp.next = None` — cut the last node off.

Trace (list is `20 → 30 → 40 → 50`):

```
temp=20: 20.next.next is 40 (not None) -> move
temp=30: 30.next.next is 50 (not None) -> move
temp=40: 40.next.next is None          -> stop
temp.next = None   ->   20 -> 30 -> 40
```

Why walk to the second-to-last node? A singly linked node only knows what comes after it. To remove the last node we must be standing on the one before it. We walk to the **second-to-last** node, then set its `next` to `None`, cutting off the last node.

Delete at a given position

```
def delete_at_position(self, position):
    if position == 0:
        self.delete_from_beginning()
        return
    temp = self.head
    for _ in range(position - 1):
        temp = temp.next
    temp.next = temp.next.next
```

Line by line:

1. `if position == 0:` — deleting the first node is a special case.
2. `self.delete_from_beginning()` / `return` — reuse that method.
3. `temp = self.head` — start at the first node.
4. `for _ in range(position - 1):` — walk to the node **just before** the one to delete.
5. `temp = temp.next` — step forward.
6. `temp.next = temp.next.next` — skip over the unwanted node by linking `temp` straight to the node after it.

Trace (list is `20 → 30 → 40`, delete position 1):

```
range(0) -> no steps, temp stays on 20
temp.next.next is 40
20.next = 40       ->   20 -> 40     (30 is skipped)
```

We walk to the node **just before** the one we want to remove, then skip over it by linking directly to the node after it.

Testing all three together

```
ll = LinkedList()
for value in [10, 20, 30, 40, 50]:
    ll.insert_at_end(value)
print("Starting list:")
ll.traverse()

ll.delete_from_beginning()
print("After deleting from beginning:")
ll.traverse()

ll.delete_from_end()
print("After deleting from end:")
ll.traverse()

ll.delete_at_position(1)
print("After deleting at position 1:")
ll.traverse()
```

Output:

```
Starting list:
10 20 30 40 50
After deleting from beginning:
20 30 40 50
After deleting from end:
20 30 40
After deleting at position 1:
20 40
```

Line by line:

1. The `for` loop uses `insert_at_end` to build `10 20 30 40 50` quickly.
2. `delete_from_beginning()` removes 10, leaving `20 30 40 50`.
3. `delete_from_end()` removes 50, leaving `20 30 40`.
4. `delete_at_position(1)` removes the node at index 1 (the 30), leaving `20 40`.

⚠ Important: no shifting, ever

Unlike arrays, deleting from a linked list **never** requires moving other elements around — we only ever change a small number of `next` references. This is the core advantage linked lists have over arrays.

References:

- Remove First Node — GeeksforGeeks
- Remove Last Node — GeeksforGeeks
- Delete at a Given Position — GeeksforGeeks

---

6. Searching

What it means: Searching means walking the list from the head, checking each node's data, until we either find a match or reach the end.

```
def search(self, target):
    temp = self.head
    position = 0
    while temp is not None:
        if temp.data == target:
            return position
        temp = temp.next
        position += 1
    return -1
```

Line by line:

1. `temp = self.head` — start at the first node.
2. `position = 0` — a counter for which index we are on. Linked lists have no built-in indexes, so we count ourselves.
3. `while temp is not None:` — look at every node until we fall off the end.
4. `if temp.data == target:` — is this the value we want?
5. `return position` — yes: report its index and stop.
6. `temp = temp.next` — no: move to the next node.
7. `position += 1` — and count one more step.
8. `return -1` — we checked every node and found nothing.

```
ll = LinkedList()
for value in [10, 20, 30, 40, 50]:
    ll.insert_at_end(value)

print(ll.search(30))
print(ll.search(99))
```

Output:

```
2
-1
```

Line by line:

1. `search(30)` finds 30 at index 2 (10 is index 0, 20 is index 1, 30 is index 2), so it prints `2`.
2. `search(99)` never matches, so the loop ends and `-1` is returned.

Notice this is the **exact same logic** as Linear Search on an array — check one at a time, in order, return the position on a match, return `-1` if we run out of nodes. The only difference is *how* we move to the next item: `temp = temp.next` instead of incrementing an index.

Reference: Search an Element in a Linked List — GeeksforGeeks

---

7. Putting It All Together

Here's the complete `LinkedList` class, combining everything from this module into one working program.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def traverse(self):
        temp = self.head
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()

    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
```

```
            return
        temp = self.head
        while temp.next is not None:
            temp = temp.next
        temp.next = new_node

    def insert_at_position(self, data, position):
        if position == 0:
            self.insert_at_beginning(data)
            return
        new_node = Node(data)
        temp = self.head
        for _ in range(position - 1):
            temp = temp.next
        new_node.next = temp.next
        temp.next = new_node

    def delete_from_beginning(self):
        if self.head is None:
            return
        self.head = self.head.next

    def delete_from_end(self):
        if self.head is None:
            return
        if self.head.next is None:
            self.head = None
            return
        temp = self.head
        while temp.next.next is not None:
            temp = temp.next
        temp.next = None

    def delete_at_position(self, position):
        if position == 0:
            self.delete_from_beginning()
            return
        temp = self.head
        for _ in range(position - 1):
            temp = temp.next
        temp.next = temp.next.next

    def search(self, target):
        temp = self.head
        position = 0
        while temp is not None:
            if temp.data == target:
                return position
            temp = temp.next
            position += 1
        return -1
```

Demo:

```
ll = LinkedList()

ll.insert_at_end(10)
ll.insert_at_end(20)
ll.insert_at_end(30)
print("Initial list:")
ll.traverse()

ll.insert_at_beginning(5)
print("After insert_at_beginning(5):")
ll.traverse()

ll.insert_at_position(15, 2)
print("After insert_at_position(15, 2):")
ll.traverse()

print("Search for 20:", ll.search(20))
print("Search for 99:", ll.search(99))

ll.delete_at_position(1)
print("After delete_at_position(1):")
ll.traverse()

ll.delete_from_end()
print("After delete_from_end():")
ll.traverse()
```

Output:

```
Initial list:
10 20 30
After insert_at_beginning(5):
5 10 20 30
After insert_at_position(15, 2):
5 10 15 20 30
Search for 20: 3
Search for 99: -1
After delete_at_position(1):
5 15 20 30
After delete_from_end():
5 15 20
```

Line by line:

1. Three `insert_at_end` calls: `10 20 30`.
2. `insert_at_beginning(5)`: `5 10 20 30`.
3. `insert_at_position(15, 2)`: `5 10 15 20 30`.
4. `search(20)` prints `3` because the list is now `5 10 15 20 30`, and 20 sits at **index 3**, not index 2. The earlier insertions moved it.
5. `search(99)` prints `-1`: not found.
6. `delete_at_position(1)` removes the 10: `5 15 20 30`.
7. `delete_from_end()` removes 30: `5 15 20`.

> 🛠️ **Instructor note (checkpoint):** Ask: *"Why did `search(20)` give 3 and not 2?"* If they can explain it, they understand how insertions change positions.

A quick note on other types of Linked Lists

Everything in this module is a **Singly** Linked List — each node only knows what comes *after* it. You may come across two other variants:

- **Doubly Linked List** — each node also has a `prev` reference, so you can walk backward too.
- **Circular Linked List** — the last node's `next` points back to the first node, instead of `None`.

We won't be covering these in depth for now — Singly Linked List is the foundation we need for what's next: **Stack** and **Queue**, which are in the second half of this module.

---

8. What is a Stack?

Stacks and Queues are both **Abstract Data Types**: they describe *what* you can do (the operations), not *how* the data is stored. In this half of the module we store the data in a chain of nodes, using the same `Node` class from Part 1, so everything you learned about `next` references applies directly.

What it means: A stack follows the rule **LIFO: Last In, First Out**. The last item you put in is the first item that comes out.

Think of a **stack of plates**. You add a plate on top, and you also take a plate from the top. You never pull one out of the middle.

Operation	What it does
`push(x)`	Put `x` on top of the stack
`pop()`	Remove and return the top item
`peek()`	Look at the top item without removing it
`is_empty()`	Check whether the stack has nothing in it

Where stacks are used: the Undo button in an editor, the Back button in a browser, and how Python keeps track of function calls.

Reference: Introduction to Stack Data Structure — GeeksforGeeks

---

9. Building a Stack

We use a linked list where the **top of the stack is the head**. Adding and removing at the head is quick, because nothing has to shift.

```
class Stack:
    def __init__(self):
        self.top = None

    def is_empty(self):
        return self.top is None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node

    def pop(self):
        if self.is_empty():
            return "Stack is empty"
        removed = self.top.data
        self.top = self.top.next
        return removed

    def peek(self):
        if self.is_empty():
            return "Stack is empty"
        return self.top.data

    def display(self):
        temp = self.top
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()
```

Line by line:

1. `class Stack:` — the blueprint for a stack.
2. `self.top = None` — `top` points at the top node. `None` means the stack is empty. (This is the same idea as `head`; we just call it `top`.)
3. `def is_empty(self):` / `return self.top is None` — gives `True` when there is nothing in the stack. We reuse this check in `pop`, `peek`.
4. `push`: `new_node = Node(data)` — build the new box.
5. `push`: `new_node.next = self.top` — **connect first:** the new node points at the old top.
6. `push`: `self.top = new_node` — **then update:** the new node is now the top.
7. `pop`: `if self.is_empty(): return "Stack is empty"` — protect against removing from an empty stack.
8. `pop`: `removed = self.top.data` — save the value *before* we lose the node.
9. `pop`: `self.top = self.top.next` — move `top` down one node. The old top is unlinked.
10. `pop`: `return removed` — give the saved value back to whoever called `pop()`.
11. `peek`: same empty check, then `return self.top.data` — read the top value **without** changing `top`.
12. `display`: `temp = self.top` then `while temp is not None:` — traversal, starting from the top and walking down.
13. `display`: `print(temp.data, end=" ")` / `temp = temp.next` / `print()` — print each value on one line, step forward, and end the line.

Trace of pushing 10, 20, 30:

```
push(10):  top -> [10]
push(20):  top -> [20] -> [10]
push(30):  top -> [30] -> [20] -> [10]
pop():     returns 30, top -> [20] -> [10]
```

🛠️ **Instructor note:** Point out that `push` is exactly `insert_at_beginning` and `pop` is exactly `delete_from_beginning`. A stack is a linked list that only ever touches the head. Ask: *"Why did we choose the head as the top and not the tail?"* (The head is the cheapest end to change. The tail would require walking the whole chain.)

How `push` works

1. `new_node = Node(data)` creates the new node.
2. `new_node.next = self.top` makes the new node point at the current top. **Connect first.**
3. `self.top = new_node` moves `top` to the new node. **Then update.**

How `pop` works

1. If the stack is empty, there is nothing to remove, so we return a message.
2. `removed = self.top.data` saves the value before we lose it.
3. `self.top = self.top.next` moves `top` down to the next node.
4. `return removed` hands back the saved value. The old node is no longer linked to anything.

Testing the stack

```
s = Stack()
s.push(10)
s.push(20)
s.push(30)
print("Stack (top first):")
s.display()
print("Peek:", s.peek())
print("Pop:", s.pop())
print("After pop:")
s.display()
print("Pop:", s.pop())
print("Pop:", s.pop())
print("Pop on empty:", s.pop())
print("Is empty:", s.is_empty())
```

Output:

```
Stack (top first):
30 20 10
Peek: 30
Pop: 30
After pop:
20 10
Pop: 20
Pop: 10
Pop on empty: Stack is empty
Is empty: True
```

Line by line:

1. `s.push(10)`, `push(20)`, `push(30)` — the stack is now `30 → 20 → 10` with 30 on top.
2. `s.display()` — prints from the top down: `30 20 10`.
3. `s.peek()` — shows 30 but does **not** remove it.
4. `s.pop()` — removes and returns 30. This is **LIFO**: 30 was last in, so it is first out.
5. `s.display()` — now `20 10`.
6. Two more `pop()` calls return 20 and 10. The stack is empty.
7. `s.pop()` on the empty stack returns the message `Stack is empty` instead of crashing.
8. `s.is_empty()` prints `True`.

Notice that 30 was pushed **last** and popped **first**. That is LIFO.

References:

- Stack - Linked List Implementation — GeeksforGeeks
- Python Program to Implement Stack Using Linked List — GeeksforGeeks

---

10. Stack Example: Reversing a Word

Because a stack gives items back in reverse order, it can reverse a word:

```
word = "PYTHON"
s = Stack()

for ch in word:
    s.push(ch)

reversed_word = ""
while not s.is_empty():
    reversed_word += s.pop()

print(reversed_word)
```

Output:

```
NOHTYP
```

Line by line:

1. `word = "PYTHON"` — the text we want to reverse.
2. `s = Stack()` — start with an empty stack.
3. `for ch in word:` — go through the word one letter at a time.
4. `s.push(ch)` — put each letter on the stack. After the loop the stack is `N → O → H → T → Y → P` with N on top.
5. `reversed_word = ""` — an empty string that will collect the answer.
6. `while not s.is_empty():` — keep going until the stack is empty.
7. `reversed_word += s.pop()` — take the top letter off and add it to the end of the string. N comes first, P comes last.
8. `print(reversed_word)` — prints `NOHTYP`.

> 🛠️ **Instructor note:** Ask: *"Why does a stack reverse things?"* (The last thing in is the first thing out, so the order flips.)

- The first loop pushes `P, Y, T, H, O, N` one by one, so `N` ends up on top.
- The second loop pops until the stack is empty, so `N` comes out first and `P` comes out last.

---

11. What is a Queue?

What it means: A queue follows the rule **FIFO: First In, First Out**. The first item that joined is the first item served.

Think of a **line at a cashier**. New people join at the back, and the person at the front is served first.

Operation	What it does
`enqueue(x)`	Add `x` at the **rear** (back)
`dequeue()`	Remove and return the **front** item
`peek()`	Look at the front item without removing it
`is_empty()`	Check whether the queue has nothing in it

Where queues are used: print jobs waiting for a printer, customers waiting for service, and tasks waiting for the CPU.

Reference: Queue in Python — GeeksforGeeks

---

12. Building a Queue

A queue touches **both ends**, so we keep two references: `front` (where we remove) and `rear` (where we add). With `rear`, adding to the back is quick, because we do not have to walk the whole chain.

```
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def is_empty(self):
        return self.front is None

    def enqueue(self, data):
        new_node = Node(data)
        if self.rear is None:
            self.front = new_node
            self.rear = new_node
            return
        self.rear.next = new_node
        self.rear = new_node

    def dequeue(self):
        if self.is_empty():
            return "Queue is empty"
        removed = self.front.data
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        return removed

    def peek(self):
        if self.is_empty():
            return "Queue is empty"
        return self.front.data

    def display(self):
        temp = self.front
        while temp is not None:
            print(temp.data, end=" ")
            temp = temp.next
        print()
```

Line by line:

1. `class Queue:` — the blueprint for a queue.
2. `self.front = None` / `self.rear = None` — two references: `front` is where we remove, `rear` is where we add. Both are `None` when the queue is empty.
3. `is_empty`: `return self.front is None` — no front means nothing in the queue.
4. `enqueue`: `new_node = Node(data)` — build the new box.
5. `enqueue`: `if self.rear is None:` — the queue is empty.
6. `enqueue`: `self.front = new_node` / `self.rear = new_node` / `return` — the first node is both the front and the rear.
7. `enqueue`: `self.rear.next = new_node` — otherwise, link the current last node to the new one.
8. `enqueue`: `self.rear = new_node` — the new node is now the last.
9. `dequeue`: `if self.is_empty(): return "Queue is empty"` — protect against an empty queue.
10. `dequeue`: `removed = self.front.data` — save the value first.
11. `dequeue`: `self.front = self.front.next` — move `front` to the second node.
12. `dequeue`: `if self.front is None: self.rear = None` — if that was the last node, the queue is empty, so `rear` must be cleared too. Otherwise `rear` would still point at a removed node.
13. `dequeue`: `return removed` — hand back the saved value.
14. `peek` and `display` — same as the Stack versions, but starting from `front`.

Trace of enqueueing 10, 20, 30 and dequeuing once:

```
enqueue(10):  front -> [10] <- rear
enqueue(20):  front -> [10] -> [20] <- rear
enqueue(30):  front -> [10] -> [20] -> [30] <- rear
dequeue():    returns 10
              front -> [20] -> [30] <- rear
```

> **Instructor note:** Ask: *"Why do we need `rear`? Could we just walk to the end each time?"* (We could, but that is slow for long queues. `rear` makes adding to the back immediate.) Then ask: *"What breaks if `dequeue` forgets to reset `rear`?"* Demonstrate with the last test line: enqueue after emptying.

How `enqueue` works

1. Create the new node.
2. **If the queue is empty**, the new node is both the `front` and the `rear`.
3. Otherwise, `self.rear.next = new_node` links it behind the current last node, and `self.rear = new_node` moves `rear` to it.

How `dequeue` works

1. If the queue is empty, return a message.
2. Save `self.front.data`, then move `front` to the next node.
3. **If `front` is now `None`, the queue just became empty**, so we also set `rear = None`. If we forget this, `rear` keeps pointing at a removed node and the next `enqueue` breaks.

Testing the queue

```
q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
print("Queue (front first):")
q.display()
print("Peek:", q.peek())
print("Dequeue:", q.dequeue())
print("After dequeue:")
q.display()
print("Dequeue:", q.dequeue())
print("Dequeue:", q.dequeue())
print("Dequeue on empty:", q.dequeue())
print("Is empty:", q.is_empty())
q.enqueue(99)
print("Enqueue after emptying, peek:", q.peek())
```

Output:

```
Queue (front first):
10 20 30
Peek: 10
Dequeue: 10
After dequeue:
20 30
Dequeue: 20
Dequeue: 30
Dequeue on empty: Queue is empty
Is empty: True
Enqueue after emptying, peek: 99
```

Line by line:

1. Three `enqueue` calls build `10 → 20 → 30`, with 10 at the front.
2. `q.display()` — prints from the front: `10 20 30`.
3. `q.peek()` — shows 10 without removing it.
4. `q.dequeue()` — removes and returns 10. This is **FIFO**: 10 joined first, so it leaves first.
5. `q.display()` — now `20 30`.
6. Two more `dequeue()` calls return 20 and 30. The queue is empty.
7. `q.dequeue()` on an empty queue returns `Queue is empty` instead of crashing.
8. `q.is_empty()` prints `True`.
9. `q.enqueue(99)` then `q.peek()` — prints 99. This works only because `dequeue` reset `rear = None` when the queue emptied.

Notice that 10 joined **first** and left **first**. That is FIFO.

References:

- Queue - Linked List Implementation — GeeksforGeeks
- Implementation of Queue using Linked List in Python — GeeksforGeeks

---

13. Queue Example: A Waiting Line

```
q = Queue()
for name in ["Ana", "Ben", "Cara"]:
    q.enqueue(name)
    print("Joined:", name)

print("Serving:", q.dequeue())
print("Serving:", q.dequeue())
print("Still waiting:")
q.display()
```

Output:

```
Joined: Ana
Joined: Ben
Joined: Cara
Serving: Ana
Serving: Ben
Still waiting:
Cara
```

Line by line:

1. `q = Queue()` — an empty waiting line.
2. `for name in ["Ana", "Ben", "Cara"]:` — go through the three names in order.
3. `q.enqueue(name)` — each person joins at the back.
4. `print("Joined:", name)` — announce who joined.
5. `print("Serving:", q.dequeue())` — the person at the front is served and removed. First call serves Ana, second serves Ben.
6. `print("Still waiting:")` / `q.display()` — shows who is left: `Cara`.

Ana joined first, so Ana is served first.

---

14. Stack vs Queue

	Stack	Queue
Rule	LIFO (Last In, First Out)	FIFO (First In, First Out)
Add	`push` on top	`enqueue` at rear
Remove	`pop` from top	`dequeue` from front
Ends used	One end (top)	Two ends (front and rear)
Real-life picture	Stack of plates	Line at a cashier
Typical use	Undo, Back button, function calls	Print jobs, waiting lines, task scheduling

Both are built on a linked list, and each of their main operations changes only the head or the tail, so none of them needs to walk or shift the whole list.

---

15. Quick Reference Cheat Sheet

Linked List

Operation	Method
Create a Node	`Node(data)`
Create an empty list	`ll = LinkedList()`
Traverse	`ll.traverse()`
Insert at beginning	`ll.insert_at_beginning(data)`
Insert at end	`ll.insert_at_end(data)`
Insert at position	`ll.insert_at_position(data, position)`
Delete from beginning	`ll.delete_from_beginning()`
Delete from end	`ll.delete_from_end()`
Delete at position	`ll.delete_at_position(position)`
Search	`ll.search(target)`

Stack and Queue

Operation	Stack	Queue
Create	`s = Stack()`	`q = Queue()`
Add	`s.push(data)`	`q.enqueue(data)`
Remove	`s.pop()`	`q.dequeue()`
Look at next item	`s.peek()`	`q.peek()`
Check empty	`s.is_empty()`	`q.is_empty()`
Show all	`s.display()`	`q.display()`

Common Beginner Mistakes — Quick Checklist

Linked List

- ✕ Forgetting to check `if self.head is None` before inserting/deleting — causes a crash on an empty list.
- ✕ Using `for` instead of `while` for traversal — we don't know the list's length in advance the way we do with an array.
- ✕ Losing the reference to the rest of the chain — always connect the new node's `next` **before** changing what points to it, or you'll disconnect part of the list.
- ✕ Confusing `self.head` (the whole list's starting point) with `temp` (a temporary variable used just to walk through the list).
- ✕ Forgetting that `position` for insertion/deletion is **0-indexed** — position 0 means "at the very beginning," same as array indexing.

Stack and Queue

- ✕ Mixing up LIFO and FIFO. Stack = plates (last in, first out). Queue = line (first in, first out).
- ✕ Popping or dequeuing without checking `is_empty()` first. This crashes on an empty structure.
- ✕ In `push`, setting `self.top = new_node` **before** `new_node.next = self.top`. The old chain is lost.
- ✕ In `dequeue`, forgetting to set `rear = None` when the queue becomes empty.
- ✕ Forgetting that `pop()` and `dequeue()` **return** the removed value. Save it in a variable or print it, or you will not see what was removed.
- ✕ Confusing `peek()` with `pop()`. Peek only looks. Pop removes.

Keep this module open in a separate tab during labs — most questions you'll run into are answered somewhere above.